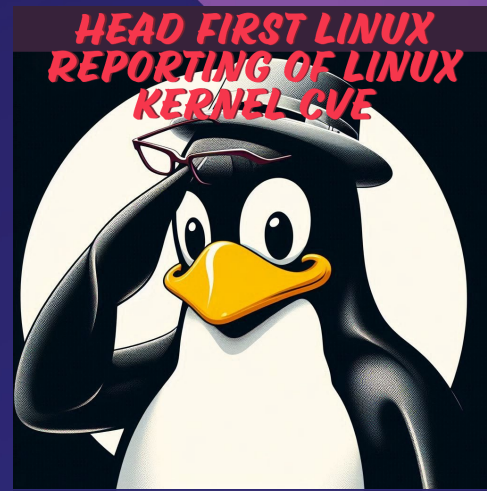




Head First Reporting of Linux Kernel CVEs : Practical Use of the Kernel Fuzzer

Yunseong Kim.
Linux Kernel CVEs reporter, Contributor



Agenda



- Linux Kernel is CNA
- Finding and Reporting Linux Kernel security bugs
 - CVE-2024-41021
 - CVE-2024-42235
- What is Kernel Fuzzer?
 - What is syzkaller?
 - Why do we need syzkaller?
 - How is Google using it?
- So, How can we fuzz it?
 - How can we reproduce it?
 - Lessons from trial and error
- Summary
- References



SOSS Community Day
JAPAN



Linux Kernel is CNA

Linux Kernel is CNA



- Linux kernel project has been accepted as a **CNA (CVE Numbering Authority)** for vulnerabilities found in Linux.
- The Linux kernel **CVE(Common Vulnerabilities and Exposures)** team does have the ability to assign CVEs for potential Linux kernel security issues.
 - <https://docs.kernel.org/process/cve.html>
- CVE numbers are allocated only after the vulnerability patches have been released.
- The CVE review process for Linux kernel security bugs is conducted from kernel.org repository
 - <https://git.kernel.org/pub/scm/linux/security/vulns.git/>



index : security/vulns.git

[master](#) ▾[switch](#)

Unnamed repository; edit this file 'description' to name the repository.

Vulns group

[about](#)[summary](#)[refs](#)[log](#)[tree](#)[commit](#)[diff](#)[stats](#)[log msg](#) ▾[search](#)

Branch	Commit message	Author	Age
master	Reject CVE-2022-48993 as it was reverted later on.	Greg Kroah-Hartman	6 hours
wip-lee	cve_review: Allow kernel version numbers when reviewing via file	Lee Jones	4 months

Age	Commit message	Author	Files	Lines
6 hours	Reject CVE-2022-48993 as it was reverted later on. HEAD master	Greg Kroah-Hartman	6	-0/+21
9 hours	update cvelistV5	Greg Kroah-Hartman	1	-0/+0
9 hours	reject CVE-2024-47755 based on review	Greg Kroah-Hartman	6	-0/+21
9 hours	reject CVE-2024-47725 as it was reverted in the same release.	Greg Kroah-Hartman	6	-0/+21
9 hours	add a .dyad entry for CVE-2024-50066	Greg Kroah-Hartman	1	-0/+5
9 hours	strip the mbox of a cve entry	Greg Kroah-Hartman	1	-11/+0
10 hours	assign CVE-2024-50066 on request	Greg Kroah-Hartman	4	-0/+243
17 hours	sasha: review v6.11.5	Sasha Levin	1	-0/+75
24 hours	add 6.11.5 cve review from greg	Greg Kroah-Hartman	1	-0/+26
25 hours	updates for latest stable releases.	Greg Kroah-Hartman	15	-12/+149
[...]				

Clone

[git://git.kernel.org/pub/scm/linux/security/vulns.git](https://git.kernel.org/pub/scm/linux/security/vulns.git)

<https://git.kernel.org/pub/scm/linux/security/vulns.git>

<https://kernel.googlesource.com/pub/scm/linux/security/vulns.git>

Age	Commit message (Expand)	Author	Files	Lines
6 hours	Reject CVE-2022-48993 as it was reverted later on. HEAD master	Greg Kroah-Hartman	6	-0/+21
9 hours	update cvelistV5	Greg Kroah-Hartman	1	-0/+0
9 hours	reject CVE-2024-47755 based on review	Greg Kroah-Hartman	6	-0/+21
9 hours	reject CVE-2024-47725 as it was reverted in the same release.	Greg Kroah-Hartman	6	-0/+21
9 hours	add a .dyad entry for CVE-2024-50066	Greg Kroah-Hartman	1	-0/+5
9 hours	strip the mbox of a cve entry	Greg Kroah-Hartman	1	-1/+0
10 hours	assign CVE-2024-50066 on request	Greg Kroah-Hartman	4	-0/+243
17 hours	sasha: review v6.11.5	Sasha Levin	1	-0/+75
24 hours	add 6.11.5 cve review from greg	Greg Kroah-Hartman	1	-0/+26
25 hours	updates for latest stable releases.	Greg Kroah-Hartman	15	-12/+149
30 hours	add .dyad files for new CVEs created	Greg Kroah-Hartman	2	-0/+19
30 hours	strip the mbox of new CVEs created.	Greg Kroah-Hartman	2	-22/+0
32 hours	assign some 2023 cve ids based on a request	Greg Kroah-Hartman	8	-0/+499
32 hours	update cvelistV5	Greg Kroah-Hartman	1	-0/+0
43 hours	strip some 2024 mbox signatures as well.	Greg Kroah-Hartman	42	-462/+0
43 hours	update cvelistV5	Greg Kroah-Hartman	1	-0/+0
43 hours	add .dyad files for newly created cve ids	Greg Kroah-Hartman	130	-0/+892
43 hours	strip signatures off some 2022 cve id messages that were just created	Greg Kroah-Hartman	88	-968/+0
43 hours	mark gsd-request-2023-01-10 as processed	Greg Kroah-Hartman	3	-0/+0
43 hours	assign cve ids based on gsd-request-2023-01-10	Greg Kroah-Hartman	352	-0/+20551
43 hours	finish reviewing gsd-request-2023-01-10	Greg Kroah-Hartman	1	-0/+89
43 hours	scripts/cve- review : add underflow to the list of things to look for	Greg Kroah-Hartman	1	-0/+1
43 hours	assign some 6.11.4 cve ids	Greg Kroah-Hartman	168	-0/+9266
44 hours	allocate some more 2024 cve ids from cve.org	Greg Kroah-Hartman	250	-0/+0
44 hours	add 6.11.4 review from greg	Greg Kroah-Hartman	1	-0/+55
44 hours	update cvelistV5	Greg Kroah-Hartman	1	-0/+0
44 hours	add dyad entries for newly created cve entries	Greg Kroah-Hartman	156	-0/+1074
44 hours	strip the mbox signatures of sent mbox messages	Greg Kroah-Hartman	157	-1727/+0
44 hours	scripts/strak: fix up error handling for no arguments	Greg Kroah-Hartman	1	-1/+5
44 hours	more 6.10.3 cve ids assigned	Greg Kroah-Hartman	64	-0/+3318
45 hours	update the 6.11.3 review from greg based on more reviews	Greg Kroah-Hartman	1	-0/+16
45 hours	assigned some 6.11.3 cve ids	Greg Kroah-Hartman	560	-0/+31694
45 hours	add first cut of greg's 6.11.3 review	Greg Kroah-Hartman	1	-0/+128
46 hours	update CVE-2024-47674 with new reference	Greg Kroah-Hartman	2	-0/+4
46 hours	CVE-2024-47674: Provide Google p0 cross-reference	Salvatore Bonaccorso	1	-0/+1
46 hours	update cvelistV5	Greg Kroah-Hartman	1	-0/+0
2 days	sasha: review v6.11.4	Sasha Levin	1	-0/+56
2 days	reject CVE-2024-47722 based on maintainer review	Greg Kroah-Hartman	6	-0/+21
2 days	add dyad entries for new CVE entries	Greg Kroah-Hartman	97	-0/+713
2 days	update mbox files for published entries	Greg Kroah-Hartman	96	-1056/+0
2 days	assign some more 6.11.2 cve ids	Greg Kroah-Hartman	20	-0/+1125
2 days	update the 6.11.2 review from greg list with some additional ones	Greg Kroah-Hartman	1	-0/+5
2 days	assign some more 6.11.2 cve ids	Greg Kroah-Hartman	32	-0/+1726

Linux Kernel is CNA

SOSS Community Day
JAPAN



- The Linux Kernel CVE review process consists of the following stages:
 - **Proposed**
 - **Done**
 - CVE Announcement

```
(base) Mac:review yunseong$ git remote -vv
origin  https://git.kernel.org/pub/scm/linux/security/vulns.git/ (fetch)
origin  https://git.kernel.org/pub/scm/linux/security/vulns.git/ (push)
(base) Mac:review yunseong$ pwd
/Users/yunseong/Documents/github/vulns/cve/review
```

```
(base) Mac:review yunseong$ ls proposed/ | sort --version-sort | head -10
v6.11.2-greg
v6.11.2-lee
v6.11.2-ruiqi
v6.11.2-sasha
v6.11.3-greg
v6.11.3-lee
v6.11.3-sasha
v6.11.4-greg
```

```
(base) Mac:review yunseong$ ls done/ | sort --version-sort --reverse | head -10
v6.11.1-sasha
v6.11.1-ruiqi
v6.11.1-lee
v6.11.1-greg
v6.10.13-ruiqi
v6.10.12-sasha
v6.10.12-ruiqi
v6.10.12-lee
v6.10.12-greg
v6.10.11-sasha
```

Linux Kernel is CNA

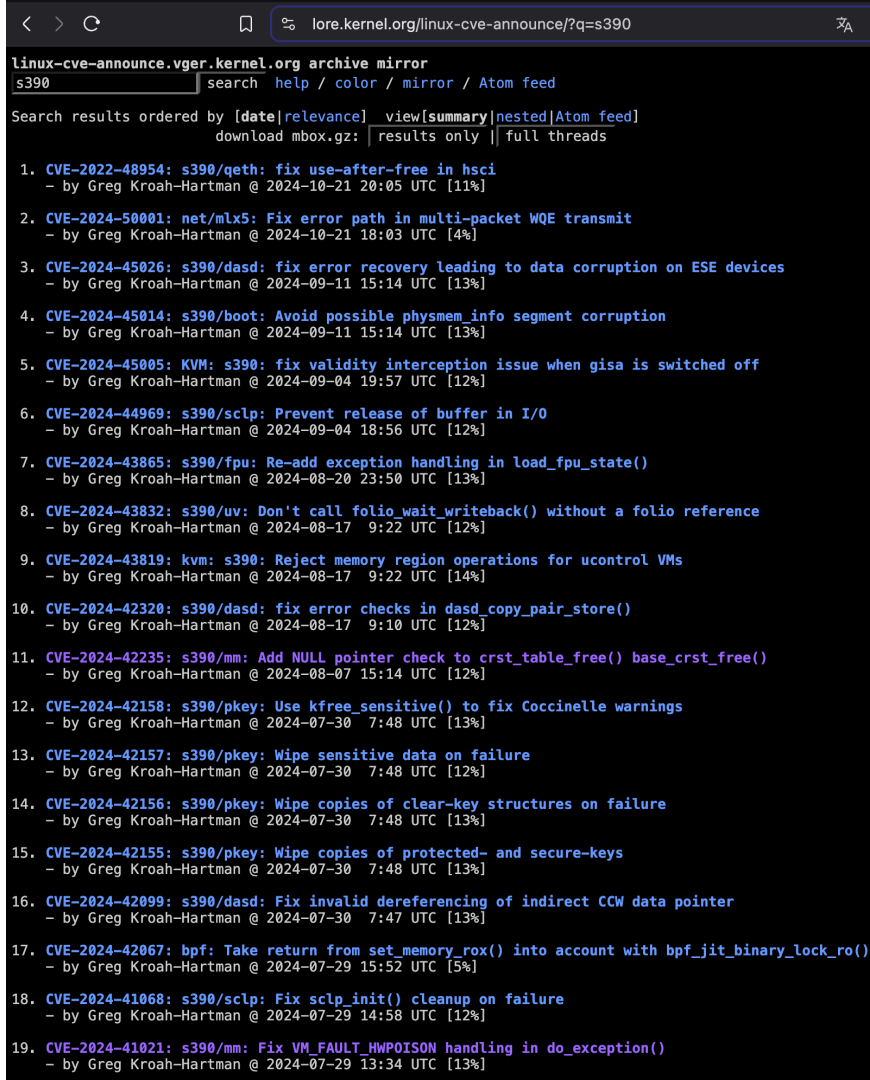
SOSS Community Day
JAPAN



- The Linux Kernel CVE review process consists of the following stages:
 - Proposed
 - Done
 - **CVE Announcement**

Linux Kernel is CNA

- Linux CVE Announce Mailing List
<https://lore.kernel.org/linux-cve-announce>
- Subscribing Digest mail from Linux CVE Announce
<mailto:linux-cve-announce+subscribe-digest@vger.kernel.org>



SOSS Community Day
JAPAN



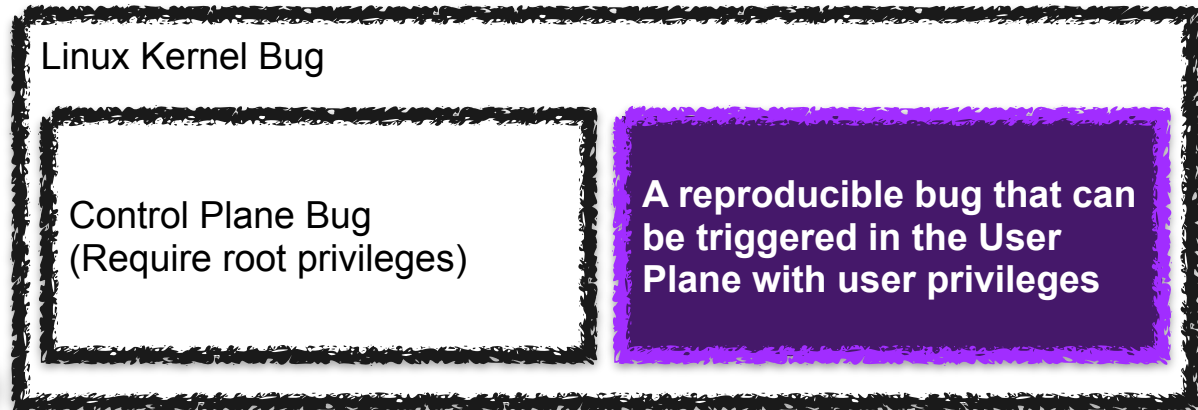
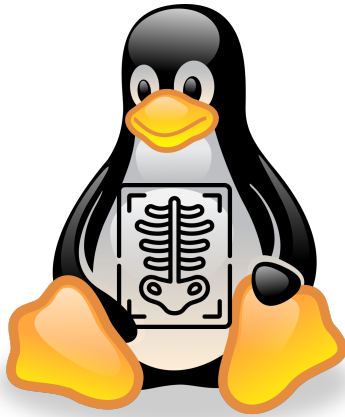
Finding and Reporting Linux Kernel security bugs

Finding and Reporting Linux Kernel security bugs

- “Think of it **like a regular patch** submission (even if you don’t have a patch yet): describe the **problem** and **impact**, list **reproduction** steps, and follow it with a **proposed fix**, all in plain text.
 - <https://docs.kernel.org/process/security-bugs.html>
- Even for someone who has experience submitting kernel patches, it can be challenging to determine the **scope** of Linux kernel security patches and what additional information should be included.
 - <https://github.com/xairy/linux-kernel-exploitation>
 - <https://github.com/google/security-research/tree/master/pocs/linux/kernelctf>

Finding and Reporting Linux Kernel security bugs

- I prepared a detailed report including the **kernel config**, **backtrace**, **reproducible code** with **user privileges**, and comprehensive **analysis** of the security bug.
- I submitted the security report directly to security@kernel.org, keeping it private as required.



CVE-2024-41021

- A Linux kernel bug related to **VM_FAULT_HWPOISON** on s390 architecture was discovered through syzkaller fuzzing, which allows unprivileged users to **crash** affecting kernel versions 6.6+ primarily on Ubuntu systems.
- CVE-2024-41021
 - <https://www.cve.org/CVERecord/?id=CVE-2024-41021>

From Yunseong Kim [@](#)

To security@kernel.org [@](#)

Bcc MichelleJin [@](#)

Subject **Re: s390/mm/fault: VM_FAULT_ERROR in do_exception() triggers kernel panic**

In reply to [<c70a8b52-51e6-496b-abda-fc51ddc33a26@gmail.com>](mailto:c70a8b52-51e6-496b-abda-fc51ddc33a26@gmail.com)

A malicious user can abuse page fault with VM_FAULT_ERROR in do_exception() This issue can trigger a s390x system crash, causing abruptly halt.

I've tested this on a s390x KVM machine(2 cpu, 8GB) in an IBM Cloud VPC, and tried to find where that 'VM_FAULT_ERROR' flag is set.

```
#define VM_FAULT_ERROR (VM_FAULT_OOM | VM_FAULT_SIGBUS | \
                        VM_FAULT_SIGSEGV | VM_FAULT_HWPoISON | \
                        VM_FAULT_HWPoISON_LARGE | VM_FAULT_FALLBACK)
```

```
do_exception()
-> handle_mm_fault()
-> handle_pte_fault()
-> __handle_mm_fault()
-> do_swap_page()
```

If FAULT_FLAG_WRITE is set in vmf->flags, execute do_wp_page(vmf)
After check bit with VM_FAULT_ERROR and set VM_FAULT_ERROR bits.

```
ret &= VM_FAULT_ERROR;
```

VM_FAULT_ERROR in vm_fault_t flag goto error label and BUG() called.

This calling 'BUG()' code has a history(Oct 3, 2002), and it looks like it needs to be updated. Whereas other architectures simply call return instead of BUG() (e.g., return; for x86, return 0; for arm64)

I found origin of the patch in the history repository in the history repository with the following commit:

s390 update (17/27): beautification

Link:

<https://git.kernel.org/pub/scm/linux/kernel/git/history/history.git/commit/arch/s390/mm/fault.c?h=v2.5.41&id=>

I also looked at the code to set VM_FAULT_ERROR to see if it would work well enough in a cloud.

Mar 5, 2008 for VM_FAULT_ERROR in do_swap_page()

```
commit: 61469f1d5177 ("memcg: when do_swap's do_wp_page fails")
```

This backtrace can be reproduced with syzkaller under user privileges.

```
kernel BUG at arch/s390/mm/fault.c:442!
monitor event: 0040 ilc:2 [#1] PREEMPT SMP
Dumping ftrace buffer:
(ftrace buffer empty)
Modules linked in:
CPU: 0 UID: 65534 PID: 16423 Comm: syz.2.1648 Not tainted
6.10.0-rc7-next-20240710-dirty #7
Hardware name: IBM 8562 GT2 A00 (KVM/Linux)
KrnL PSW : 0704e00180000000 000003ffe00812bc (do_exception+0x1674/0x1730
```

From [Linus Torvalds](#) 

To [Yunseong Kim](#) 

Cc [security@kernel.org](#) , [Alexander Gordeev](#) , [Gerald Schaefer](#) , [Heiko Carstens](#) , [Vasily Gorbik](#) , [Christian Bornträger](#) , [Sven Schnelle](#) 

Subject **Re: s390/mm/fault: VM_FAULT_ERROR in do_exception() triggers kernel panic**

In reply to [<dc109a32-ea04-43b1-95ac-1386c32d1b48@gmail.com>](#)

I have forwarded the original report to the s390 people. Please keep everybody on the participants list if you have more information.

On Fri, 12 Jul 2024 at 15:32, Yunseong Kim [yskelg@gmail.com](mailto:<yskelg@gmail.com>) wrote:

```
do_exception()
-> handle_mm_fault()
-> handle_pte_fault()
-> __handle_mm_fault()
-> do_swap_page()

If FAULT_FLAG_WRITE is set in vmf->flags, execute do_wp_page(vmf)
After check bit with VM_FAULT_ERROR and set VM_FAULT_ERROR bits.

ret &= VM_FAULT_ERROR;
```

Note: that doesn't set any VM_FAULT_ERROR bits. It only clears the non-error bits.

I suspect that the bit that is set is VM_FAULT_HWPOISON, maybe coming from HWPOISON_INJECT, maybe from some other source. I didn't actually try to narrow it down.

What was your reproducer to cause the fault bits to be set, just for interest?

But yes, at least x86 just considers VM_FAULT_HWPOISON to be a SIGBUS event, although it does try to then print out a bit more information about where the hw poisoning happened.

I suspect the minimal patch is something like this (whitespace-damaged - only meant as a possible suggestion):

```
--- a/arch/s390/mm/fault.c
+++ b/arch/s390/mm/fault.c
@@ -433,13 +433,11 @@ static void do_exception(struct pt_regs
*regs, int access)
{
    handle_fault_error_nolock(regs, 0);
    else
        do_sigsegv(regs, SEGV_MAPERR);
- } else if (fault & VM_FAULT_SIGBUS) {
+ } else {
    if (!user_mode(regs))
        handle_fault_error_nolock(regs, 0);
    else
        do_sigbus(regs);
- } else {
-     BUG();
- }
}
```

but I will leave it to the s390 people to decide.

From Gerald Schaefer 

To Linus Torvalds , Yunseong Kim 

Cc security@kernel.org , Alexander Gordeev , Heiko Carstens , Vasily Gorbik , Christian Bornträger , Sven Schnelle , Gerald Schaefer 

Subject **[PATCH] s390/mm: fix VM_FAULT_HWPoISON handling in do_exception()**

In reply to 20240715195155.7e43e8bb@thinkpad-T15

There is no support for HWPoISON, MEMORY_FAILURE, or ARCH_HAS_COPY_MC on s390. Therefore we do not expect to see VM_FAULT_HWPoISON in do_exception().

However, since commit af19487f00f3 ("mm: make PTE_MARKER_SWAPIN_ERROR more general"), it is possible to see VM_FAULT_HWPoISON in combination with PTE_MARKER_POISONED, even on architectures that do not support HWPoISON otherwise. In this case, we will end up on the BUG() in do_exception().

Fix this by treating VM_FAULT_HWPoISON the same as VM_FAULT_SIGBUS, similar to x86 when MEMORY_FAILURE is not configured. Also print unexpected fault flags, for easier debugging.

Note that VM_FAULT_HWPoISON_LARGE is not expected, because s390 cannot support swap entries on other levels than PTE level.

Cc: stable@vger.kernel.org # 6.6+
Fixes: af19487f00f3 ("mm: make PTE_MARKER_SWAPIN_ERROR more general")
Reported-by: Yunseong Kim syskelg@gmail.com
Signed-off-by: Gerald Schaefer gerald.schaefer@linux.ibm.com

```
---
arch/s390/mm/fault.c | 3 +-
1 file changed, 2 insertions(+), 1 deletion(-)

diff --git a/arch/s390/mm/fault.c b/arch/s390/mm/fault.c
index 65747f15dbec..c848966ed175 100644
--- a/arch/s390/mm/fault.c
+++ b/arch/s390/mm/fault.c
@@ -433,12 +433,13 @@ static void do_exception(struct pt_regs *regs, int access)
        handle_fault_error_nolock(regs, 0);
    } else
        do_sigsegv(regs, SEGV_MAPERR);
-   } else if (fault & VM_FAULT_SIGBUS) {
+   } else if (fault & (VM_FAULT_SIGBUS | VM_FAULT_HWPoISON)) {
        if (!user_mode(regs))
            handle_fault_error_nolock(regs, 0);
        else
            do_sigbus(regs);
+   } else {
+       pr_emerg("Unexpected fault flags: %08x\n", fault);
+       BUG();
    }
}
```

2.40.1

From Yunseong Kim [📧](#)

To Alexander Gordeev [📧](#), Gerald Schaefer [📧](#)

Cc Linus Torvalds [📧](#), security@kernel.org [📧](#), Heiko Carstens [📧](#), Vasily Gorbik [📧](#), Christian Bornträger [📧](#), Sven Schnelle [📧](#)

Subject **Re: [PATCH] s390/mm: fix VM_FAULT_HWPOISON handling in do_exception()**

In reply to [<ZpVmlCzkGsAdW7m@li-008a6a4c-3549-11b2-a85c-c5cc2836eea2.ibm.com>](mailto:CzkGsAdW7m@li-008a6a4c-3549-11b2-a85c-c5cc2836eea2.ibm.com)

Acked-by: Alexander Gordeev [<agordeev@linux.ibm.com>](mailto:agordeev@linux.ibm.com)

Thank you Gerald. After applying the patch, I can confirm that even after running thousands of tests, no panic occurs. Before that, panic occurred within 30 times of tests.

What was your reproducer to cause the fault bits to be set, just for interest?

Also interested, also which kernel config was used.

My goal for this fuzzing was to contribute more meaningfully to the s390x Linux kernel. This config based on arch/s390/config :

```
make debug_defconfig
make kvm_guest.config
```

To add, I need to update syzkaller's documents to follow the current state of s390x Linux kernel.

Link:
https://github.com/google/syzkaller/blob/master/docs/linux/setup_linux-host_qemu-vm_s390x-kernel.md#kernel

To make things a bit easier, I've attached the script I updated, 'syzconf.sh'. Based on the config generated from the guide.

I've attached the 'config-6.10.0-next-20240715-fuzz' file, which has been a few items more menuconfigured for stable s390x syzkaller fuzzing.

There is no support for HWPOISON, MEMORY_FAILURE, or ARCH_HAS_COPY_MC on s390. Therefore we do not expect to see VM_FAULT_HWPOISON in do_exception().

I'm a little confused though, I don't see VM_FAULT_HWPOISON in my config file, but I haven't had any panics with the reproducer since I applied Gerald's patch.

Although I was unable to analyze it precisely, I sincerely appreciate everyone taking the time to look into it so carefully despite busy schedule.

If there's anything else someone need me to check, please let me know. I'll check it out.

Tested-by: Yunseong Kim [<syskelg@gmail.com>](mailto:syskelg@gmail.com)

Warm regards,

Yunseong Kim

↩ Rep

From Gerald Schaefer 

To Yunseong Kim , Linus Torvalds , Greg KH 

Cc Alexander Gordeev , security@kernel.org , Heiko Carstens , Vasily Gorbik , Christian Bornträger , Sven Schnelle , Gerald Schaefer 

Subject **Re: [PATCH] s390/mm: fix VM_FAULT_HWPOISON handling in do_exception()**

In reply to [<1d3a72e9-6353-47ca-b244-a6912f5a82c5@gmail.com>](mailto:1d3a72e9-6353-47ca-b244-a6912f5a82c5@gmail.com)

On Tue, 16 Jul 2024 18:09:58 +0900
Yunseong Kim [<yskelg@gmail.com>](mailto:yskelg@gmail.com) wrote:

[...]

There is no support for HWPOISON, MEMORY_FAILURE, or ARCH_HAS_COPY_MC on s390. Therefore we do not expect to see VM_FAULT_HWPOISON in do_exception().

I'm a little confused though, I don't see VM_FAULT_HWPOISON in my config file, but I haven't had any panics with the reproducer since I applied Gerald's patch.

The config option that would normally result in possible VM_FAULT_HWPOISON flag in fault handler is CONFIG_MEMORY_FAILURE, which we do not support on s390, and it is also not set in your config.

As explained in my patch description, this is a different case, not depending on any specific config options. After commit af19487f00f3, it is possible to see the VM_FAULT_HWPOISON in fault handler also by using UFFD with UFFDIO_POISON.

Your syzkaller run must have hit this scenario. We were now also able to reproduce the crash with the uffd-unit-tests from the kernel source selftests, also as unprivileged user.

Linus, Greg, how important do you consider being able to crash the kernel, as unprivileged user, in terms of security vs. normal bug? Given that "normally" users do not login on our s390 systems we don't think this is too important (plus it is "only" denial of service).

This should affect v6.6+, so no current s390 RHEL/SLES release, but Ubuntu would be affected. How would you suggest to proceed, can this be handled as normal bug fix?

[...]

Tested-by: Yunseong Kim [<yskelg@gmail.com>](mailto:yskelg@gmail.com)

Thanks a lot for reporting and testing!

Gerald

From Greg KH [🔗](#)

To Gerald Schaefer [🔗](#)

Cc Yunseong Kim [🔗](#), Linus Torvalds [🔗](#), Alexander Gordeev [🔗](#), security@kernel.org [🔗](#), Heiko Carstens [🔗](#), Vasily Gorbik [🔗](#), Christian Bornträger [🔗](#), Sven Schnelle [🔗](#)

Subject **Re: [PATCH] s390/mm: fix VM_FAULT_HWPOISON handling in do_exception()**

In reply to [<20240716183452.2151ae72@thinkpad-T15>](#)

On Tue, Jul 16, 2024 at 06:34:52PM +0200, Gerald Schaefer wrote:

On Tue, 16 Jul 2024 18:09:58 +0900

Yunseong Kim yskelg@gmail.com wrote:

[...]

There is no support for HWPOISON, MEMORY_FAILURE, or ARCH_HAS_COPY_MC on s390. Therefore we do not expect to see VM_FAULT_HWPOISON in do_exception().

I'm a little confused though, I don't see VM_FAULT_HWPOISON in my config file, but I haven't had any panics with the reproducer since I applied Gerald's patch.

The config option that would normally result in possible VM_FAULT_HWPOISON flag in fault handler is CONFIG_MEMORY_FAILURE, which we do not support on s390, and it is also not set in your config.

As explained in my patch description, this is a different case, not depending on any specific config options. After commit af19487f00f3, it is possible to see the VM_FAULT_HWPOISON in fault handler also by using UFFD with UFFDIO_POISON.

Your syzkaller run must have hit this scenario. We were now also able to reproduce the crash with the uffd-unit-tests from the kernel source selftests, also as unprivileged user.

Linus, Greg, how important do you consider being able to crash the kernel, as unprivileged user, in terms of security vs. normal bug? Given that "normally" users do not login on our s390 systems we don't think this is too important (plus it is "only" denial of service).

This should affect v6.6+, so no current s390 RHEL/SLES release, but Ubuntu would be affected. How would you suggest to proceed, can this be handled as normal bug fix?

Normal bug fix is fine, but note that it will be getting a CVE 😊

thanks,

greg k-h

From: Greg Kroah-Hartman <gregkh@linuxfoundation.org>
To: linux-cve-announce@vger.kernel.org
Cc: Greg Kroah-Hartman <gregkh@linuxfoundation.org>
Subject: [CVE-2024-41021: s390/mm: Fix VM_FAULT_HWPoISON handling in do_exception\(\)](#)
Date: Mon, 29 Jul 2024 15:34:29 +0200 [\[thread overview\]](#)
Message-ID: <2024072929-CVE-2024-41021-f857@gregkh> (raw)

Description

In the Linux kernel, the following vulnerability has been resolved:

s390/mm: Fix VM_FAULT_HWPoISON handling in do_exception()

There is no support for HWPoISON, MEMORY_FAILURE, or ARCH_HAS_COPY_MC on s390. Therefore we do not expect to see VM_FAULT_HWPoISON in do_exception().

However, since commit af19487f00f3 ("mm: make PTE_MARKER_SWAPIN_ERROR more general"), it is possible to see VM_FAULT_HWPoISON in combination with PTE_MARKER_POISONED, even on architectures that do not support HWPoISON otherwise. In this case, we will end up on the BUG() in do_exception().

Fix this by treating VM_FAULT_HWPoISON the same as VM_FAULT_SIGBUS, similar to x86 when MEMORY_FAILURE is not configured. Also print unexpected fault flags, for easier debugging.

Note that VM_FAULT_HWPoISON_LARGE is not expected, because s390 cannot support swap entries on other levels than PTE level.

The Linux kernel CVE team has assigned CVE-2024-41021 to this issue.

Affected and fixed versions

Issue introduced in 6.6 with commit af19487f00f3 and fixed in 6.9.12 with commit 9e13767ccef
Issue introduced in 6.6 with commit af19487f00f3 and fixed in 6.10.2 with commit a3aefb871222
Issue introduced in 6.6 with commit af19487f00f3 and fixed in 6.11-rc1 with commit df39038cd895

Please see <https://www.kernel.org> for a full list of currently supported kernel versions by the kernel community.

Unaffected versions might change over time as fixes are backported to older supported kernel versions. The official CVE entry at <https://cve.org/CVERecord/?id=CVE-2024-41021> will be updated if fixes are backported, please check that for the most up to date information about this issue.

Affected files

The file(s) affected by this issue are:
arch/s390/mm/fault.c

Mitigation

The Linux kernel CVE team recommends that you update to the latest stable kernel version for this, and many other bugfixes. Individual changes are never tested alone, but rather are part of a larger kernel release. Cherry-picking individual commits is not recommended or supported by the Linux kernel community at all. If however, updating to the latest release is impossible, the individual changes to resolve this issue can be found at these commits:

<https://git.kernel.org/stable/c/9e13767ccefdc4f8aa92514b592b60f6b54882ff>
<https://git.kernel.org/stable/c/a3aefb871222a9880602d1a44a558177b4143e3b>
<https://git.kernel.org/stable/c/df39038cd89525d465c2c8827eb64116873f141a>

CVE-2024-42235

- NULL pointer dereference bug in s390's *crst_table_free()* was discovered and fixed by adding NULL pointer checks, which could occur during error handling in *crst_table_upgrade()* when **FAIL_PAGE_ALLOC** is enabled.
- CVE-2024-42235
 - <https://nvd.nist.gov/vuln/detail/CVE-2024-42235>

From Yuseong Kim [@](#)
To security@kernel.org [@](#)
Bcc MichelleJin [@](#)
Subject s390/pgalloc: kernel pointer dereference during huge memory unmapping

In 'crst_table_upgrade()', The pointer 'p4d' is set to 'NULL'.

If the 'asce_limit' condition isn't met (asce_limit != _REGION2_SIZE) and memory allocation using 'pgd = crst_table_alloc(mm)' fails, going to the 'err_pgd' label, which might attempt to call 'crst_table_free(mm, p4d)'.

However, if 'p4d' is still NULL due to the failed 'pgd' allocation, dereferencing it would result in a kernel pointer dereference error.

If the user map/unmap the huge memory is frequent in the s390x arch, This vulnerability is more likely to be reproduced.

I've tested this on a s390x KVM machine(2 cpu, 8GB) in an IBM Cloud VPC.
Linux paran 6.10.0-rc6-next-20240703 #5 SMP PREEMPT Sat Jul 6 22:47:12 UTC 2024 s390x s390x s390x GNU/Linux

I think the solution is like other architectures (e.g. x86) that call the 'pagetable_free()', put a value check in when calling crst_table_free().

Below is a backtrace that can be reproduced with syzkaller.

Unable to handle kernel pointer dereference in virtual kernel address space
Failing address: 009c006c7000000 TED: 009c006c7000803
Fault in home space mode while using kernel ASCE.

AS:0000000006b3c00b R2:000000007e7b000b R3:0000000006b45007

S:0000000006b44000 P:00000000006b51315

Oops: 0038 ilc:2 [#1] PREEMPT SMP

Dumping ftrace buffer:

(ftrace buffer empty)

Modules linked in:

CPU: 0 UID: 0 PID: 50527 Comm: syz.1.2721 Not tainted

6.10.0-rc6-next-20240703 #5

Hardware name: IBM 8562 GT2 A00 (KVM/Linux)

Krnl PSW : 0704f00180000000 000003ffe0b36012 (bytes_is_nonzero

mm/kasan/generic.c:86 [inline])

Krnl PSW : 0704f00180000000 000003ffe0b36012 (memory_is_nonzero

mm/kasan/generic.c:104 [inline])

Krnl PSW : 0704f00180000000 000003ffe0b36012 (memory_is_poisoned_n

mm/kasan/generic.c:129 [inline])

Krnl PSW : 0704f00180000000 000003ffe0b36012 (memory_is_poisoned

mm/kasan/generic.c:161 [inline])

Krnl PSW : 0704f00180000000 000003ffe0b36012 (check_region_inline

mm/kasan/generic.c:180 [inline])

Krnl PSW : 0704f00180000000 000003ffe0b36012

(kasan_check_range+0x222/0x2f8 mm/kasan/generic.c:189)

R:0 T:1 IO:1 EX:1 Key:0 M:1 W:0 P:0 AS:3 CC:3 PM:0 RI:0 EA:3

Krnl GPRS: 001c000000000001 009c006c7000000 0400036638000000

0000000000000008

0000000000000000 000003ffe009490e 0080006c7000000

0000000000000001

0400036638000007 009c006c7000001 009c006c7000000

009c006c7000000

000003ffe3ec3f88 000003ffca3f9587 000003fe1cbf7b0

000003fe1cbf748

Krnl Code: 000003ffe0b36006: 4197b000

la %r9,0(%r7,%r11)

000003ffe0b3600a: a7710001

tmll

Krnl Code: 000003ffe0b36006: 4197b000
000003ffe0b3600a: a7710001
#000003ffe0b3600e: a784000f
>000003ffe0b36012: 9500b000
000003ffe0b36016: a7740023
000003ffe0b3601a: a51c001c
000003ffe0b3601e: a51b0001
000003ffe0b36022: b9e81016

la %r9,0(%r7,%r11)
tmll %r7,1
brc 8,000003ffe0b3602c
cli 0(%r11),0
brc 7,000003ffe0b3605c
llihh %r1,28
oill %r1,1
agrk %r1,%r6,%r1

Call Trace:

[<000003ffe0b36012>] bytes_is_nonzero mm/kasan/generic.c:86 [inline]
[<000003ffe0b36012>] memory_is_nonzero mm/kasan/generic.c:104 [inline]
[<000003ffe0b36012>] memory_is_poisoned_n mm/kasan/generic.c:129 [inline]
[<000003ffe0b36012>] memory_is_poisoned mm/kasan/generic.c:161 [inline]
[<000003ffe0b36012>] check_region_inline mm/kasan/generic.c:180 [inline]
[<000003ffe0b36012>] kasan_check_range+0x222/0x2f8 mm/kasan/generic.c:189
[<000003ffe009490e>] instrument_atomic_read
include/linux/instrumented.h:68 [inline]
[<000003ffe009490e>] _test_bit
include/asm-generic/bitops/instrumented-non-atomic.h:141 [inline]
[<000003ffe009490e>] compound_order include/linux/mm.h:1107 [inline]
[<000003ffe009490e>] pagetable_free include/linux/mm.h:2886 [inline]
[<000003ffe009490e>] crst_table_free+0x6e/0x130 arch/s390/mm/pgalloc.c:58
[<000003ffe0094e16>] crst_table_upgrade+0x446/0x518
arch/s390/mm/pgalloc.c:131
[<000003ffe0085460>] check_asce_limit
arch/s390/include/asm/pgalloc.h:45 [inline]
[<000003ffe0085460>] arch_get_unmapped_area_topdown+0x6b0/0x870
arch/s390/mm/mmap.c:167
[<000003ffe009fe0a8>] arch_get_unmapped_area_topdown_vmflags+0x48/0x58
mm/mmap.c:1877
[<000003ffe09fe1ac>] mm_get_unmapped_area_vmflags+0xf4/0x120 mm/mmap.c:1887
[<000003ffe0b5eb62>] __thp_get_unmapped_area mm/huge_memory.c:878 [inline]
[<000003ffe0b5eb62>] thp_get_unmapped_area_vmflags+0x1c2/0x338
mm/huge_memory.c:911
[<000003ffe09fe4a6>] __get_unmapped_area+0x2ce/0x328 mm/mmap.c:1927
[<000003ffe0a097fc>] do_mmap+0x314/0xe40 mm/mmap.c:1325
[<000003ffe0945b5a>] vm_mmap_pgoff+0x1da/0x3d0 mm/util.c:588
[<000003ffe09fc814>] ksys_mmap_pgoff+0x3e4/0x518 mm/mmap.c:1484
[<000003ffe09fcbc8>] __do_sys_old_mmap mm/mmap.c:1517 [inline]
[<000003ffe09fcbc8>] __se_sys_old_mmap mm/mmap.c:1508 [inline]
[<000003ffe09fcbc8>] _s390x_sys_old_mmap+0x130/0x178 mm/mmap.c:1508
[<000003ffe000f08e>] do_syscall+0x336/0x468 arch/s390/kernel/syscall.c:117
[<000003ffe3359d6c>] __do_syscall+0xb4/0x198 arch/s390/kernel/syscall.c:141
[<000003ffe3394c98>] system_call+0x70/0x98 arch/s390/kernel/entry.S:284

INFO: lockdep is turned off.

Last Breaking-Event-Address:

[<000003ffe0b35e4a>] memory_is_nonzero mm/kasan/generic.c:103 [inline]
[<000003ffe0b35e4a>] memory_is_poisoned_n mm/kasan/generic.c:129 [inline]
[<000003ffe0b35e4a>] memory_is_poisoned mm/kasan/generic.c:161 [inline]
[<000003ffe0b35e4a>] check_region_inline mm/kasan/generic.c:180 [inline]
[<000003ffe0b35e4a>] kasan_check_range+0x5a/0x2f8 mm/kasan/generic.c:189

Best regards,

Yuseong Kim

From Linus Torvalds 

To Yunseong Kim 

Cc security@kernel.org , Heiko Carstens , Vasily Gorbik , Alexander Gordeev , Christian Bornträger , Sven Schnelle 

Subject **Re: s390/pgalloc: kernel pointer dereference during huge memory unmapping**

In reply to <16253cb4-f700-44b9-b761-36cfcbc64ed3@gmail.com>

On Mon, 8 Jul 2024 at 14:28, Yunseong Kim [yskelg@gmail.com](mailto:<yskelg@gmail.com>) wrote:

In 'crst_table_upgrade()', The pointer '*p4d' is set to 'NULL'.

If the 'asce_limit' condition isn't met (asce_limit != _REGION2_SIZE) and memory allocation using 'pgd = crst_table_alloc(mm)' fails, going to the 'err_pgd' label. which might attempt to call 'crst_table_free(mm, p4d)'.

However, if 'p4d' is still NULL due to the failed 'pgd' allocation, dereferencing it would result in a kernel pointer dereference error.

I suspect this counts as just a "regular bug" rather than anything else, but I've forwarded the original report to s390 people, and they are cc'd here too.

Please keep everybody on the participants list if there is further comments or questions about this.

Linus

From Alexander Gordeev [✉](#)
To Heiko Carstens [✉](#)
Cc Linus Torvalds [✉](#), Yunseong Kim [✉](#), security@kernel.org [✉](#), Vasily Gorbik [✉](#), Christian Bornträger [✉](#), Sven Schnelle [✉](#)
Subject **Re: s390/pgalloc: kernel pointer dereference during huge memory unmapping**
In reply to [<20240709124520.7263-C-hca@linux.ibm.com>](#)

On Tue, Jul 09, 2024 at 02:45:20PM +0200, Heiko Carstens wrote:

Hi Heiko,

```
...
From c1a8786a1c8447e0570624e30f9c4847ab368a06 Mon Sep 17 00:00:00 2001
From: Heiko Carstens <hca@linux.ibm.com>
Date: Tue, 9 Jul 2024 08:50:56 +0200
Subject: [PATCH] s390/mm: Add NULL pointer check to crst_table_free() /
base_crst_free()

crst_table_free() used to work with NULL pointers before the conversion to
ptdescs. Since crst_table_free() can be called with a NULL pointer (error
handling in crst_table_upgrade() add an explicit check.
Also add the same check to base_crst_free() for consistency reasons.

In real life this should not happen, since order two GFP_KERNEL allocations
will not fail, unless FAIL_PAGE_ALLOC is enabled and used.

Reported-by: Yunseong Kim <syskel@gmail.com>
Fixes: 6326c26c1514 ("s390: convert various pgalloc functions to use ptdescs")
Signed-off-by: Heiko Carstens <hca@linux.ibm.com>
---
arch/s390/mm/pgalloc.c | 4 ++++
1 file changed, 4 insertions(+)

diff --git a/arch/s390/mm/pgalloc.c b/arch/s390/mm/pgalloc.c
index abb629d7e131..7e3e767ab87d 100644
--- a/arch/s390/mm/pgalloc.c
+++ b/arch/s390/mm/pgalloc.c
@@ -55,6 +55,8 @@ unsigned long *crst_table_alloc(struct mm_struct *mm)

void crst_table_free(struct mm_struct *mm, unsigned long *table)
{
+   if (!table)
+       return;
   pagetable_free(virt_to_ptdesc(table));
}

@@ -262,6 +264,8 @@ static unsigned long *base_crst_alloc(unsigned long val)

static void base_crst_free(unsigned long *table)
{
+   if (!table)
+       return;
   pagetable_free(virt_to_ptdesc(table));
}
```

Acked-by: Alexander Gordeev [<agordeev@linux.ibm.com>](#)

From Yunseong Kim 

To Linus Torvalds , Heiko Carstens 

Cc security@kernel.org , Vasily Gorbik , Alexander Gordeev , Christian Bornträger , Sven Schnelle 

Bcc MichelleJin 

Subject **Re: s390/pgalloc: kernel pointer dereference during huge memory unmapping**

In reply to <CAHk=whY+b9DdiLbDXpbYEkAv+zVg39Kk2ez-TF9AVG4LfrfVA@mail.gmail.com>

Hello,

On 7/9/24 11:26 오후, Linus Torvalds wrote:

On Tue, 9 Jul 2024 at 05:45, Heiko Carstens hca@linux.ibm.com> wrote:

Linus, should this come as a regular pull request, or would you just apply the patch? Right now we do not have any other fixes pending for 6.10.

I can take it directly if that's more convenient. It's th last week of the 6.10 release, and if you have nothing else pending I guess just taking it as a direct patch is fine.

Linus

I want to express my sincere gratitude to all for the great help.

After applying Heiko's patch, I conducted rigorous testing on user permissions with the same scenario.

Pleasingly, I haven't encountered the issue since applying the patch, even after executing the test over 10 million times.

Prior to applying the patch, the kernel panic would occur within a few thousand runs in user permissions.

Thank you again everyone for the response despite busy schedules.

Tested-by: Yunseong Kim [yskelg@gmail.com](mailto:<yskelg@gmail.com>)>

Best regards,

Yunseong Kim

From: Greg Kroah-Hartman <gregkh@linuxfoundation.org>
To: linux-cve-announce@vger.kernel.org
Cc: Greg Kroah-Hartman <gregkh@linuxfoundation.org>
Subject: CVE-2024-42235: s390/mm: Add NULL pointer check to crst_table_free() base_crst_free()
Date: Wed, 7 Aug 2024 17:14:41 +0200 [thread overview]
Message-ID: <2024080740-CVE-2024-42235-efae@gregkh> (raw)

Description

=====

In the Linux kernel, the following vulnerability has been resolved:

s390/mm: Add NULL pointer check to crst_table_free() base_crst_free()

crst_table_free() used to work with NULL pointers before the conversion to ptdescs. Since crst_table_free() can be called with a NULL pointer (error handling in crst_table_upgrade()) add an explicit check.

Also add the same check to base_crst_free() for consistency reasons.

In real life this should not happen, since order two GFP_KERNEL allocations will not fail, unless FAIL_PAGE_ALLOC is enabled and used.

The Linux kernel CVE team has assigned CVE-2024-42235 to this issue.

Affected and fixed versions

=====

Issue introduced in 6.6 with commit 6326c26c1514 and fixed in 6.6.41 with commit 794fa52b9463
Issue introduced in 6.6 with commit 6326c26c1514 and fixed in 6.9.10 with commit f80bd8bb6f38
Issue introduced in 6.6 with commit 6326c26c1514 and fixed in 6.10 with commit b5efb63acf7b

Please see <https://www.kernel.org> for a full list of currently supported kernel versions by the kernel community.

Unaffected versions might change over time as fixes are backported to older supported kernel versions. The official CVE entry at <https://cve.org/CVERecord?id=CVE-2024-42235>

will be updated if fixes are backported, please check that for the most up to date information about this issue.

Affected files

=====

The file(s) affected by this issue are:
arch/s390/mm/pgalloc.c

Mitigation

=====

The Linux kernel CVE team recommends that you update to the latest stable kernel version for this, and many other bugfixes. Individual changes are never tested alone, but rather are part of a larger kernel release. Cherry-picking individual commits is not recommended or supported by the Linux kernel community at all. If however, updating to the latest release is impossible, the individual changes to resolve this issue can be found at these commits:

<https://git.kernel.org/stable/c/794fa52b94637d6b2e8c9474fbc3983af5c9f046>
<https://git.kernel.org/stable/c/f80bd8bb6f380bc265834c46058d38b34174813e>
<https://git.kernel.org/stable/c/b5efb63acf7bddaf20eacfcac654c25c446eabe8>

SOSS Community Day
JAPAN



What is Kernel Fuzzer?

What is Kernel Fuzzer?

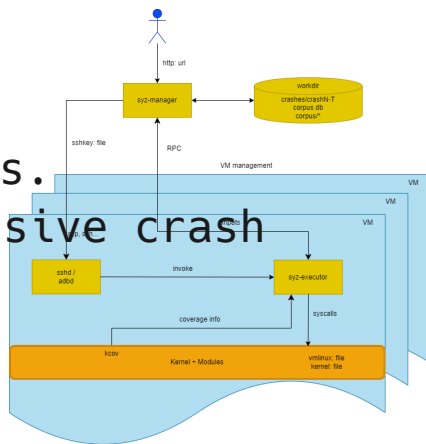
- The Linux kernel uses system calls to connect user and kernel space, and handles device communications through network and hardware protocols.
- These communication channels are tested using automated tools designed to find bugs in the Linux kernel through various manipulations.
- Coverage-guided fuzzing that uses KCOV(/sys/kernel/debug/kcov) to track which code paths are executed.
 - <https://docs.kernel.org/dev-tools/kcov.html>
- Utilizing Kernel Debugging Features and [Sanitizers](#).

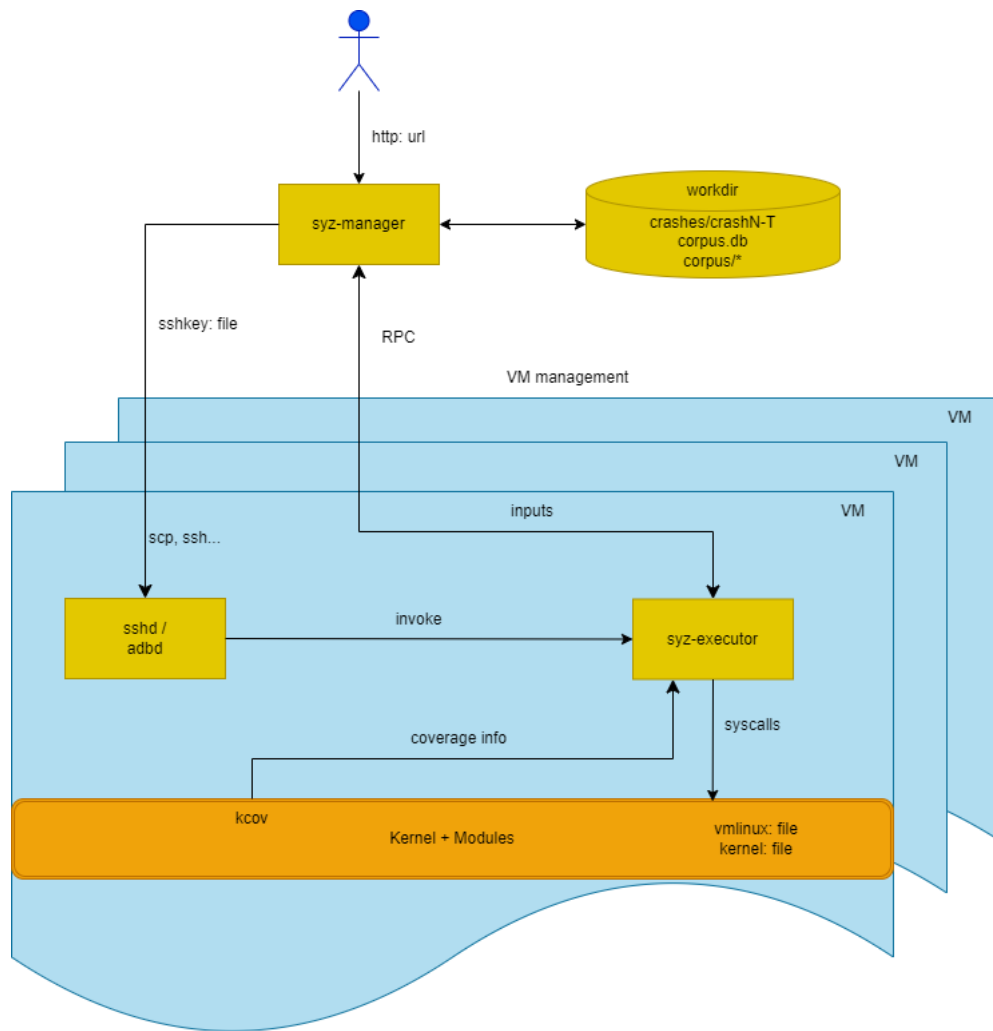
What is syzkaller?

SOSS Community Day
JAPAN



- Automated Kernel fuzzing tool
- Generates/mutates test programs based on interface descriptions
 - <https://github.com/google/syzkaller/tree/master/sys/linux>
- Executes syscall sequences
 - Programs are randomly generated based on the mentioned descriptions
- Collects and mutates coverage-guided corpus
 - Mutations include inserting and
 - removing syscalls and changing their arguments.
- Combines kernel debugging features with comprehensive crash analysis feedback





tmux: server,2172

└bash,70863

└─syz-manager,184674 -config /home/paran/syzkaller/workdir/ksmbd.cfg

└─qemu-system-aar,186236 -m 4096 -smp 3 -chardev socket,id=SOCKSYZ,ser

└─{qemu-system-aar},186238

└─{qemu-system-aar},186242

└─{qemu-system-aar},186243

└─{qemu-system-aar},186244

└─{qemu-system-aar},186245

└─{qemu-system-aar},186248

└─{qemu-system-aar},186249

└─{qemu-system-aar},186265

└─{qemu-system-aar},186271

└─{qemu-system-aar},186283

└─{qemu-system-aar},186288

└─{qemu-system-aar},186289

└─{syz-manager},184675

└─{syz-manager},184676

└─{syz-manager},184677

└─{syz-manager},184678

└─{syz-manager},184679

└─{syz-manager},184691

└─{syz-manager},184692

└─{syz-manager},184693

└─{syz-manager},184696

└─{syz-manager},184700

└─{syz-manager},184701

└─{syz-manager},184740

└─sshd,936

└─sshd,1216

└─syz-executor,1274 runner 0 localhost 44537

└─syz-executor,6236 exec

└─syz-executor,6237 exec

└─syz-executor,6240 exec

└─syz-executor,6241 exec

└─syz-executor,6246 exec

└─syz-executor,6248 exec

└─syz-executor,6247 exec

└─syz-executor,6250 exec

└─syz-executor,6258 exec

└─syz-executor,6267 exec

```
{
```

```
"name": "QEMU-aarch64",  
"target": "linux/arm64",  
"http": ":56701",  
"sshkey": "/path/.ssh/id_ed25519",  
"syzkaller": "/path/syzkaller",  
"image": "/path/ubuntu.img",  
"procs": 3,  
"max_crash_logs": 100,  
"sandbox": "setuid",  
"cover": true,  
"cover_filter": {  
    "files": ["^fs/smb/"],  
    "functions": ["^ksmbd"]  
},  
"reproduce": false,  
"type": "qemu",  
"vm": {  
    "count": 1,  
    "cpu": 4, "mem": 4096,  
    "qemu": "/path/qemu-system-aarch64",  
    "cmdline": "quiet splash oops=panic panic_on_warn=1 panic=-1 ftrace_dump_on_oops=orig_cpu slub_debug=UZ nokaslr",  
    "kernel": "/path/linux/arch/arm64/boot/Image",  
    "qemu_args": "-machine virt -enable-kvm -cpu host",  
    "initrd": "/path/initrd.img"  
},  
"enable_syscalls": ["syz_ksmbd_send_req"],
```

```
}
```

SOSS Community Day
JAPAN



Why do we need syzkaller?
(Product Stability)

SOSS Community Day
JAPAN



- Early detection and fixes of bugs in kernel modules and custom kernels help improve overall product stability.
- Supports user mode fuzzing operations
 - **"sandbox": "setuid"** mode running as user 'nobody'
 - **"sandbox": "namespace"** mode requiring CONFIG_USER_NS kernel support

Why do we need syzkaller? (Product Stability)



- Enables proactive cyber defense for Linux kernel.
 - During product development, identify and fix vulnerabilities that could be exploited for malicious attacks, including memory bugs (use-after-free, double-free) and race conditions.
- The EU's Cyber Resilience Act (CRA):

Companies must report security incidents to ENISA within 24 hours of becoming aware of them and implement resolution measures. Failure to comply can result in fines of up to €15 million or 2.5% of annual global turnover.

<https://digital-strategy.ec.europa.eu/en/library/cyber-resilience-act>

Why do we need syzkaller? (Open Source Contribution)



- From a kernel contributor perspective
 - "sandbox": "none" mode running as user 'root' also welcome! This is default option and gives the most coverage.
- Verify if this bug has already been addressed in subsystem patches or discussed in the mailing lists.
- It involves identifying clear bugs from dumps, collaborating with the community to analyze them, and receiving thorough reviews and appreciation from subsystem maintainers through contributions.

Why do we need syzkaller?
(Make for Fun!)

SOSS Community Day
JAPAN



- Can be used on both virtual machines and actual boards.
 - https://github.com/google/syzkaller/blob/master/docs/linux/setup_linux-host_isolated.md
 - `/sys/fs/pstore CONFIG_PSTORE=y, CONFIG_PSTORE_RAM=y`

How is Google using it?

- How is google doing it?
- syzbot upstream
 - <https://syzkaller.appspot.com/upstream>
 - Wait, Does Google cover all the different versions of the Linux kernel?
 - There are so many open issues
- syzbot mailing List
 - <https://groups.google.com/g/syzkaller>
 - Linux kernel maintainers keep a close watch on it!

syzbot

Linux



Open [1308]



Subsystems



Fixed [5747]



Invalid [14067]



Misc



Send us feedback

The fixing commit must reach the following trees ([more info](#)):

URL	Branch
git://git.kernel.org/pub/scm/linux/kernel/git/arm64/linux.git	for-kernelci
git://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git	master
git://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf.git	master
git://git.kernel.org/pub/scm/linux/kernel/git/davem/net-next.git	main
git://git.kernel.org/pub/scm/linux/kernel/git/davem/net.git	main
git://git.kernel.org/pub/scm/linux/kernel/git/kvmarm/kvmarm.git	fuzzme
git://git.kernel.org/pub/scm/linux/kernel/git/next/linux-next.git	master
git://git.kernel.org/pub/scm/linux/kernel/git/riscv/linux.git	fixes
git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git	master
https://git.kernel.org/pub/scm/linux/kernel/git/gregkh/usb.git	usb-testing

Instances [tested repos]:

Name	Last active	Uptime	Corpus	Coverage 	Crashes	Execs	Kernel build			Status	syzkaller build			Bugs	
							Commit	Config	Freshness		Commit	Freshness	Status	All	Only
ci-qemu-gce-upstream-auto	now	22h44m	11842	171252	161	280238	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-qemu-native-arm64-kvm	now	1d13h	980	16698		50516	38ba0d2e5c2e	.config	4d22h		65e8686b	1d21h		all	only
ci-qemu-upstream	now	22h38m	38404	654516	488	60365	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-qemu-upstream-386	now	23h07m	40915	620439	369	346510	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-qemu2-arm32	now	23h07m	127646	142765	176	205535	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-qemu2-arm64	now	22h26m	79068	93066	57	116658	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-qemu2-arm64-compatible	now	22h43m	99171	115959	76	110445	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-qemu2-arm64-mte	now	22h20m	176112	194422	34	134266	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-qemu2-riscv64	now	1d11h	26274	387925	42	96362	5f153a692bac	.config	1d20h		65e8686b	1d21h		all	only
ci-snapshot-upstream-root	now	22h43m	62752	331079	641	921711	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-bpf-kasan-gce	now	1d18h	20008	187454	601	242735	ae90f6a6170d	.config	2d10h		65e8686b	1d21h		all	only
ci-upstream-bpf-next-kasan-gce	now	1d16h	21124	190082	972	306677	87cb58aebdf7	.config	1d17h		65e8686b	1d21h		all	only
ci-upstream-gce-arm64	now	1d18h	66920	547685	1725	broken	7678c1b2735a	.config	2d15h		65e8686b	1d21h		all	only
ci-upstream-gce-leak	now	23h26m	47243	819915	1055	983021	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-kasan-badwrites-root	now	23h54m	40991	803601	102	199448	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-kasan-gce	now	23h43m	49013	408722	389	561770	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-kasan-gce-386	now	23h30m	47107	402265	161	351886	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-kasan-gce-root	now	1d00h	69915	680315	484	452551	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-kasan-gce-selinux-root	now	1d01h	39366	682253	357	634316	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-kasan-gce-smack-root	now	1d00h	71774	592053	453	707293	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-kmsan-gce-386-root	now	1d01h	60730	478721	119	201052	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-kmsan-gce-root	now	1d00h	68657	515901	353	363901	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci-upstream-linux-next-kasan-gce-root	5d20h					broken	15e7d45e786a	.config	11d	failing	65e8686b	1d21h		all	only
ci-upstream-net-kasan-gce	now	1d17h	41684	280069	438	526545	6d858708d465	.config	11d	failing	65e8686b	1d21h		all	only
ci-upstream-net-this-kasan-gce	now	1d17h	24084	132867	807	1326936	e31a8219fbfc	.config	2d00h		65e8686b	1d21h		all	only
ci2-upstream-fs	1d08h					broken	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci2-upstream-kcsan-gce	now	1d08h	82513	500153	820	1148397	850925a8133c	.config	1d11h		65e8686b	1d21h		all	only
ci2-upstream-usb	now	1d08h	3341	83161	138	582320	c6d9e43954bf	.config	6d03h		65e8686b	1d21h		all	only

open (1099):

Title	Repro	Cause bisect	Fix bisect	Count	Last	Reported	Discussions
possible deadlock in process_measurement (4) integrity lsm	C	done		8171	now	29d	PATCH [4d16h]
KCSAN: assert: race in dequeue_entities audit				1893	now	29d	🗨 12 [2d22h]
possible deadlock in lock_mm_and_find_vma (2) mm	C			6102	now	57d	🗨 1 [23d]
possible deadlock in ocfs2_init_acl ocfs2	C			1834	now	66d	
possible deadlock in ocfs2_try_remove_refcount_tree ocfs2	C	done		30916	now	68d	🗨 1 [3d00h]
kernel BUG in zero_user_segments ntfs3	C	inconclusive		9360	now	72d	
WARNING: refcount bug in j1939_session_put can	C	done		6681	now	82d	PATCH [15d]
possible deadlock in team_del_slave (3) net	C	error		199936	now	183d	PATCH [112d]
KASAN: slab-use-after-free Read in _ethtool_get_link_ksettings net	syz	done		1888	now	218d	PATCH [201d]
WARNING in folio_mark_dirty (2) gfs2	C	error		559772	now	606d	🗨 4 [23d]
WARNING in cm109_urb_irq_callback/usb_submit_urb usb input	C	unreliable		436	now	1397d	PATCH [1133d]
BUG: workqueue lockup (5) net	C	done		8082	now	1747d	
KASAN: slab-out-of-bounds Read in dbAllocBits jfs	C			51	5m	170d	
unregister_netdevice: waiting for DEV to become free (8) net	C	done		25979	5m	505d	PATCH [257d]
INFO: rcu detected stall in newfstatat (3) usb	C	done		139	6m	504d	🗨 4 [233d]
possible deadlock in ocfs2_reserve_suballoc_bits ocfs2	C	done		2646	8m	69d	
KASAN: invalid-free in dev_free usb	C			1066	8m	71d	
INFO: task hung in iterate_supers 9p	C	error		4419	8m	2300d	PATCH [2297d]
KASAN: vmalloc-out-of-bounds Write in imageblit (4) fbdev	C			122	9m	66d	
KMSAN: uninit-value in hfs_brec_find hfs	C			189	10m	689d	PATCH [358d]
kernel BUG in jfs_evict_inode jfs	C	inconclusive		22928	10m	761d	
kernel BUG in _bch2_trans_commit bcache/hfs	C	unreliable		3306	11m	6d04h	
KASAN: slab-use-after-free Read in hdm_disconnect usb	C			8526	11m	89d	PATCH [89d]
WARNING in page_pool_put_unrefed_netmem net				3	12m	61d	
KMSAN: uninit-value in _exfat_get_dentry_set exfat	C			11	13m	21d	🗨 7 [8d03h]
possible deadlock in do_ip_setsockopt (4) net	C	done		2537	13m	122d	PATCH [34d]
possible deadlock in smc_vlan_by_tcpsk net s390	C			620	16m	122d	PATCH [33d]
UBSAN: array-index-out-of-bounds in dbAdjTree (2) jfs	C	inconclusive		57	16m	161d	PATCH [18d]
WARNING in _ieee80211_beacon_get wireless	C	done		34463	16m	1482d	🗨 6 [13d]
KASAN: slab-use-after-free Read in percpu_ref_put (2) block ...	C			86	18m	16d	
INFO: trying to register non-static key in ocfs2_dlm_shutdown ocfs2	C			72	18m	39d	
WARNING: suspicious RCU usage in dev_deactivate_queue net	C	error		4645	18m	101d	🗨 1 [23d]
KMSAN: uninit-value in eth_type_trans (2) net	C			5708	19m	1739d	🗨 5 [23d]
WARNING in _rate_control_send_low (3) wireless				88	21m	19h11m	
KASAN: out-of-bounds Read in ext4_xattr_set_entry ext4	C			134	23m	35d	PATCH [9d04h]
possible deadlock in smc_switch_to_fallback (2) net s390	C	error		3897	23m	124d	
kernel BUG in hfs_write_inode hfs	C	error		22933	24m	701d	

So, How can we fuzz it?

- Build your kernel with debug configuration and sanitizer
 - When I checked recently, Either KASAN (Almost architecture supported) or KCSAN (x86 or Arm64) or KMSAN (x86 + Clang ≥ 14 only)
- Prepare QEMU (KVM enabled host CPU preferred)
- Prepare Disk
 - root files system from Yocto Framework
 - Well known distribution img with initrd
- ssh-copy-id with private key

So, How can we fuzz it?

- There are useful debug config in fedora's machines
- <https://src.fedoraproject.org/rpms/kernel/raw/rawhide/f/kernel-aarch64-debug-fedora.config>
- syzbot config

ci-upstream-kasan-gce-386	now	1d00h	47156	402342	187	401809	850925a8133c	.config
ci-upstream-kasan-gce-root	now	1d02h	70138	680628	554	516628	850925a8133c	.config
ci-upstream-kasan-gce-selinux-root	now	1d02h	39529	682566	415	723685	850925a8133c	.config
ci-upstream-kasan-gce-smack-root	now	1d02h	71926	592387	519	809738	850925a8133c	.config
ci-upstream-kmsan-gce-386-root	now	1d02h	60944	479094	131	228290	850925a8133c	.config
ci-upstream-kmsan-gce-root	now	1d01h	68801	516097	400	421486	850925a8133c	.config



QEMU-s390x syzkaller [config] d7906eff enable expert mode

CAPTION: Stats

```

candidates 0
corpus      15030
coverage    314382
exec total  333321 (23/min)
fuzzing VMs 1
pending     0
reproducing 0
crash types 159
crashes     2897
suppressed  0
syscalls    2092
uptime      839843 sec
    
```

CAPTION: Crashes:

	Description	Count	Last Time	Report
BUG:	Bad page state	49	2024/08/12 20:39	
BUG:	Bad rss-counter state	81	2024/08/30 09:40	
BUG:	key ADDR has not been registered!	83	2024/07/17 23:14	
BUG:	using smp_processor_id() in preemptible code in event_function_call	200	2024/08/17 23:58	
BUG:	workqueue lockup	233	2024/09/18 22:38	
	can't ssh into the instance	9	2024/10/27 04:04	
	failed to read from qemu: EOF	256	2024/10/15 15:11	
INFO:	task hung in __lru_add_drain_all	12	2024/08/24 05:21	
INFO:	task hung in addrconf_dad_work	1	2024/07/26 20:24	
INFO:	task hung in cleanup_net	1	2024/07/26 20:30	
INFO:	task hung in collapse_huge_page	10	2024/07/16 13:41	
INFO:	task hung in linkwatch_event	3	2024/08/23 05:07	
INFO:	task hung in perf_event_release_kernel	1	2024/08/17 08:26	
INFO:	task hung in ppp_exit_net	1	2024/08/14 20:25	

BUG: using smp_processor_id() in preemptible code in event_function_call

#	Log Report	Time Tag
---	------------	----------

67	log report	
----	------------	--

2024/08/17	23:58	
------------	-------	--

66	log report	
----	------------	--

2024/08/17	23:14	
------------	-------	--

65	log report	
----	------------	--

2024/08/17	23:09	
------------	-------	--

64	log report	
----	------------	--

2024/08/17	22:35	
------------	-------	--

63	log report	
----	------------	--

2024/08/17	22:30	
------------	-------	--

62	log report	
----	------------	--

2024/08/17	21:36	
------------	-------	--

61	log report	
----	------------	--

2024/08/17	21:26	
------------	-------	--

60	log report	
----	------------	--

2024/08/17	21:22	
------------	-------	--

59	log report	
----	------------	--

2024/08/17	21:18	
------------	-------	--

58	log report	
----	------------	--

2024/08/17	21:13	
------------	-------	--

57	log report	
----	------------	--

2024/08/17	21:09	
------------	-------	--

56	log report	
----	------------	--

2024/08/17	21:04	
------------	-------	--

55	log report	
----	------------	--

BUG: using smp_processor_id() in preemptible [00000000] code: syz.4.5/2019
caller is event_function_call+0xd8/0x588 kernel/events/core.c:301
CPU: 1 UID: 65534 PID: 2019 Comm: syz.4.5 Not tainted 6.11.0-rc3-next-20240814 #1
Hardware name: IBM 8562 GT2 A00 (KVM/Linux)

Call Trace:

```
[<000003ffe33aca1e>] __dump_stack lib/dump_stack.c:94 [inline]
[<000003ffe33aca1e>] dump_stack_lvl+0x27e/0x348 lib/dump_stack.c:120
[<000003ffe3449f60>] check_preemption_disabled+0x100/0x110 lib/smp_processor_id.c:49
[<000003ffe08341b8>] event_function_call+0xd8/0x588 kernel/events/core.c:301
[<000003ffe086c4ae>] perf_remove_from_context+0x106/0x1e8 kernel/events/core.c:2488
[<000003ffe086f422>] perf_event_release_kernel+0x16a/0x970 kernel/events/core.c:5474
[<000003ffe086fc74>] perf_release+0x4c/0x68 kernel/events/core.c:5568
[<000003ffe0c1c3b8>] __fput+0x3a8/0xb10 fs/file_table.c:424
[<000003ffe01a6c84>] task_work_run+0x1ac/0x2c0 kernel/task_work.c:228
[<000003ffe036f02a>] resume_user_mode_work+0x8a/0x90 include/linux/resume_user_mode.h:50
[<000003ffe036f9b4>] exit_to_user_mode_loop kernel/entry/common.c:114 [inline]
[<000003ffe036f9b4>] exit_to_user_mode_prepare include/linux/entry-common.h:328 [inline]
[<000003ffe036f9b4>] __syscall_exit_to_user_mode_work kernel/entry/common.c:207 [inline]
[<000003ffe036f9b4>] syscall_exit_to_user_mode_work+0x1dc/0x2d0 kernel/entry/common.c:212
[<000003ffe34480cc>] __do_syscall+0xbc/0x1b8 arch/s390/kernel/syscall.c:141
[<000003ffe346ce34>] system_call+0x74/0x98 arch/s390/kernel/entry.S:293
```

1 lock held by syz.4.5/2019:

#0: 000002f28e3c02b8 (&ctx->mutex){+..}-{3:3}, at: perf_event_ctx_lock_nested+0x27c/0x578 kernel/events/core.c:1308
sctp: Hash tables configured (bind 32/51)

[illegible]

How can we reproduce it?

- Reproduce with log (syzlang) scripts
 - `$ ./syz-execprog -executor=./syz-executor -repeat=0 -sandbox=setuid -enable=binfmt_misc,cgroups,close_fds,devlink_pci,ieee802154,net_dev,net_reset,nic_vf,swap,sysctl,tun,usb,vhci,wifi -procs 8 log0`
- Additionally, these can be converted into C source code.
 - `$ ./syz-prog2c -prog log0 -repeat=1 -enable=none > reproducible.c`
- Filter testing to focus on specific system calls or devices.

Lessons from trial and error



- Use [tmux](#) to maintain persistent fuzzing sessions
- Latest compiler versions recommended for optimal sanitizer features
- Enable KVM acceleration with QEMU arguments:
 - cpu host -enable kvm
- Sufficient storage space for data partition
 - Minimum 16GB data partition size
 - When fuzzing, additional tools or services may be required to write descriptions for specific subsystems.
 - Latest compiler toolchain
 - Dependencies
 - Support utilities

Lessons from trial and error



- Set reproduce = false in syzkaller config file. If not, syzkaller will use crash log (list of programs executed), to perform reproducing step, to find the program that causes the crash. We can ignore time-consuming.
- Set sandbox = setuid -> **executor/common_linux.h**, `do_sandbox_setuid()`
 - Fuzzing as user nobody = 65534 but not root.
 - Crashes caused by root have little/no meaning for CVE.
- If you find a bug, running it as root with none mode privileges is sufficient.
- Do with enable/disable system(also, pseudo system call) call
- cover filter file with regex or pcs
 - pcs(Program Counters)
 - `clang -emit-llvm -c kerne.c -o kern.ll && objdump -d vmlinux > kern.asm`
 - `0xffffffff81000000:0x10\n`
- If you want to fuzzing your new kernel program, you need to create syzlang descriptions for it and generate the necessary files for syzkaller support.
 - https://github.com/google/syzkaller/blob/master/docs/syscall_descriptions.md

```

# MBZ except for SMB3 or later
    Channel          int32
    RemainingBytes    int32
    ReadChannelInfoOffset  offsetof[Buffer, int16]
    ReadChannelInfoLength len[Buffer, int16]
    Buffer            smbd_buffer_descriptor_v1
} [packed]

_union_smb2_req [
    smb2_negotiate_req    smb2_negotiate_req
    smb2_sess_setup_req   smb2_sess_setup_req
    smb2_echo_req         smb2_echo_req
    smb2_read_req         smb2_read_req
] [varlen]

smb2_req {
    req_base      _union_smb2_req
    req_andx1     optional[_union_smb2_req]
    req_andx2     optional[_union_smb2_req]
} [packed]

smb2_req_pdu {
    _pdu_size     len[req, int32be]
    req           smb2_req
} [packed]

```

```

syz_ksmbd_send_req(req_packet ptr[in, smb2_req_pdu], req_len len[req_packet], res_packet buffer[out], res_len len[res_packet])

```

- <https://github.com/google/syzkaller/tree/master/sys/linux>

```

/*#if SYZ_EXECUTOR 1) ...*/
#include <netinet/in.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>

#define KSMBD_BUF_SIZE 16000

static long syz_ksmdb_send_req(volatile long a0, volatile long a1, volatile long a2, volatile long a3)
{
    int packet_reqlen;
    int errno;
    struct sockaddr_in serv_addr;
    char packet_req[KSMBD_BUF_SIZE]; // max frame size

    debug("[*]{syz_ksmdb_send_req} entered ksmdb send...\n");

    if (a0 == 0 || a1 == 0) {
        debug("[*]{syz_ksmdb_send_req} param empty\n");
        return -7;
    }

    sockfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (sockfd < 0) {
        debug("[*]{syz_ksmdb_send_req} failed to create socket\n");
        return -1;
    }

    memset(&serv_addr, '0', sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    serv_addr.sin_port = htons(445);

    errno = connect(sockfd, (struct sockaddr*)&serv_addr, sizeof(serv_addr));
    if (errno < 0) {
        debug("[*]{syz_ksmdb_send_req} failed to connect (err: %d)\n", errno);
        return errno ^ 0xff80000;
    }

    // prepend kcov handle to packet
    packet_reqlen = a1 + 8 > KSMBD_BUF_SIZE ? KSMBD_BUF_SIZE - 8 : a1;
    *(unsigned long*)packet_req = procid + 1;
    memcpy(packet_req + 8, (char*)a0, packet_reqlen);

    if (write(sockfd, (char*)packet_req, packet_reqlen + 8) < 0)
        return -4;

    if (read(sockfd, (char*)a2, a3) < 0)
        return -5;

    if (close(sockfd) < 0)
        return -6;

    debug("[*]{syz_ksmdb_send_req} successfully returned\n");

    return 0;
}
#endif

```

- https://github.com/google/syzkaller/blob/master/executor/common_linux.h

```
diff --git a/pkg/vminfo/linux_syscalls.go b/pkg/vminfo/linux_syscalls.go
index 95e3e54eb..4351c4fa4 100644
--- a/pkg/vminfo/linux_syscalls.go
+++ b/pkg/vminfo/linux_syscalls.go
@@ -103,8 +103,14 @@ var linuxSyscallChecks = map[string]func(*checkContext, *prog.Syscall) string{
     "syz_socket_connect_nvme_tcp": linuxSyzSocketConnectNvmeTCPSupported,
     "syz_pidfd_open":              alwaysSupported,
     "syz_create_resource":         alwaysSupported,
+    "syz_ksmbd_send_req":         alwaysSupported,
 }
```

- https://github.com/google/syzkaller/blob/master/pkg/vminfo/linux_syscalls.go

Lessons from trial and error

SOSS Community Day
JAPAN



- Let's review what we covered at the beginning.
- “Think of it like a regular patch submission (even if you don’t have a patch yet): describe the **problem** and **impact**, list **reproduction** steps, and follow it with a **proposed fix**, all in plain text.
 - <https://docs.kernel.org/process/security-bugs.html>
- Even for someone who has experience submitting kernel patches, it can be challenging to determine the **scope** of Linux kernel security patches and what additional information should be included.
 - <https://github.com/xairy/linux-kernel-exploitation>
 - <https://github.com/google/security-research/tree/master/pocs/linux/kernelctf>

From Yunseong Kim 
To security@kernel.org , cve@kernel.org 
Cc Yeoreum Yun , Yunseong Kim , Yunseong Kim <yunseong.kim@ericsson.com> 
Subject **[PATCH] qdisc: Fix null-ptr-deref from perf_trace_qdisc_reset**

From: Yunseong Kim <yskelg@gmail.com>

During qdisc initialization, qdisc was being set to noop_queue.
In veth_init_queue, the initial tx_num was reduced back to one,
causing the qdisc reset to be called with noop, which led to the panic.

This patch resolve Null dereference from
qdisc->dev_queue->dev <NULL> ->name situation.

Signed-off-by: Yunseong Kim <yskelg@gmail.com>

include/trace/events/qdisc.h | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

```
diff --git a/include/trace/events/qdisc.h b/include/trace/events/qdisc.h
index f1b5e816e7e5..170b51f8e47a 100644
--- a/include/trace/events/qdisc.h
+++ b/include/trace/events/qdisc.h
@@ -81,7 +81,7 @@ TRACE_EVENT(qdisc_reset,
     TP_ARGS(q),

     TP_STRUCT__entry(
-        __string(      dev,          qdisc_dev(q)->name      )
+        __string(dev, qdisc_dev(q) ? qdisc_dev(q)->name : "noop_queue")
         __string(      kind,         q->ops->id              )
         __field(       u32,          parent                  )
         __field(       u32,          handle                  )
     )

```

2.45.2

This is not the right way to
report vulnerabilities.



From Greg KH 
To Yunseong Kim 
Cc security@kernel.org , cve@kernel.org , Yeoreum Yun , Yunseong Kim <yunseong.kim@ericsson.com> 
Subject **Re: [PATCH] qdisc: Fix null-ptr-deref from perf_trace_qdisc_reset**
In reply to <20240619200951.5999-1-yskelg@gmail.com>

On Thu, Jun 20, 2024 at 05:09:51AM +0900, yskelg@gmail.com wrote:

From: Yunseong Kim yskelg@gmail.com

include/trace/events/qdisc.h | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

This is not how to submit patches, please just send them to the proper developers and mailing lists involved.

From Yunseong Kim <yskelg@gmail.com>
To security@kernel.org, cve@kernel.org
Cc Yeoreum Yun <ppbuk5246@gmail.com>, Yunseong Kim <yskelg@gmail.com>, Levi Yun <yeoreum.yun@arm.com>
Subject [PATCH] hci: fix double free in hci_req_sync

From: Yunseong Kim <yskelg@gmail.com>

The approach taken to address the 'CVE-2024-35978' introduced another double-free vulnerability. commit 45d355a926ab
("Bluetooth: Fix memory leak in hci_req_sync_complete()")

'hdev->req_skb' double free scenario:

cpu1	cpu2
====	====
sock_ioctl	
sock_do_ioctl	
hci_sock_ioctl	
hci_dev_cmd	
hci_req_sync	
__hci_req_sync	hci_rx_work
kfree_skb(hdev->req_skb)	hci_event_packet
(sleep)	hci_req_sync_complete
__ Longer times,	kfree_skb(hdev->req_skb)
reproduce well	hdev->req_skb = NULL

The longer cpu1 sleep in '__hci_req_sync', the more reproducible it is. We've tested it by inserting the 'msleep()' function, and it's frequently at 1000ms, and It has been consistently reproducible at 2000ms.

We confirmed the detection with various workloads that cause CPU1 to sleep. The call trace below is one of the KASAN has seen.

Bluetooth: hci0: unexpected cc 0x0c38 length: 249 > 2

=====

BUG: KASAN: slab-use-after-free in skb_release_data+0x7d8/0x8a0 home/paran/linux/net/core/skbuff.c:1119
Write of size 1 at addr ffff0000947d3fe by task syz-executor.2/2010

CPU: 1 PID: 2010 Comm: syz-executor.2 Not tainted
6.10.0-rc4-00217-g35bb670d65fc-dirty #22 Hardware name: linux,dummy-virt (DT)
Call trace:
dump_backtrace+0x318/0x348 home/paran/linux/arch/arm64/kernel/stacktrace.c:317
show_stack+0x4c/0x80 home/paran/linux/arch/arm64/kernel/stacktrace.c:324
__dump_stack home/paran/linux/lib/dump_stack.c:88 [inline]
dump_stack_lvl+0x214/0x328 home/paran/linux/lib/dump_stack.c:114
print_address_description home/paran/linux/mm/kasan/report.c:377 [inline]
print_report+0x2ac/0x948 home/paran/linux/mm/kasan/report.c:488
kasan_report+0xc8/0x148 home/paran/linux/mm/kasan/report.c:601
__asan_report_store1_noabort+0x44/0x60 home/paran/linux/mm/kasan/report_generic.c:383
skb_release_data+0x7d8/0x8a0 home/paran/linux/net/core/skbuff.c:1119

This is not the right way to report vulnerabilities.



From Greg KH 
To Yunseong Kim 
Cc security@kernel.org , cve@kernel.org , Yeoreum Yun , Yeoreum Yun 
Subject **Re: [PATCH] hci: fix double free in hci_req_sync**
In reply to <20240701182147.96747-2-yskelg@gmail.com>

On Tue, Jul 02, 2024 at 03:21:48AM +0900, 'Yunseong Kim wrote:

| **From: Yunseong Kim** [<yskelg@gmail.com>](mailto:yskelg@gmail.com)

<snip>

cve@kernel.org is NOT some place to send any patches, ESPECIALLY security patches, as it has nothing to do with fixing things, as per our documentation.

Please just work with the Bluetooth developers normally, now that this is not really a "private" report at all.

tahnks,

greg k-h

Summary

SOSS Community Day
JAPAN



- How to properly report kernel security bugs
- Helpful tips for effective kernel fuzzing with syzkaller

References



- Posting

- <https://lwn.net/Articles/961961/>
- <https://lwn.net/Articles/677764/>
- <https://xairy.io/articles/syzkaller-external-network>
- <https://github.com/google/syzkaller/pull/3245/files>
- <https://googleprojectzero.blogspot.com/2017/05/exploiting-linux-kernel-via-packet.html>
- <https://www.cyberark.com/resources/threat-research-blog/your-nvme-had-been-syzed-fuzzing-nvme-of-tcp-driver-for-linux-with-syzkaller>
- <https://github.com/hardenedlinux>
- <https://gist.github.com/paranlee/a8c1db153eaaabfe1ba8133e651952a9>
- <https://gist.github.com/paranlee/b7157d9bec33ae10198380aa0af7e77f>
- https://groups.google.com/g/syzkaller/c/8ZI_sY90JEw

- Video

- <https://youtu.be/HeeoTE9jLjM?si=xvV7k6FKxq6fsBex>
- https://youtu.be/Rg_VPMT0XXw?si=06YwZBFYdLguyjGG
- <https://youtu.be/Jc25CM1Ppgo?si=aE7sTSg3fV0u50i0>
- <https://youtu.be/HN2qFpIvATk?si=WENKzWj9ae-ZFHjX>
- https://youtu.be/amEMG63ebQM?si=_Y6eRwmOd3L1Jnif
- <https://youtu.be/oB4mdLycRnQ?si=HN011JqVt6WFHa6C>
- https://youtu.be/JK_oGcebEg8?si=Twhw6wUX0r62ECQg
- https://youtu.be/sDMNEBoTtrI?si=Kc2_tXIN2A25-kdT

SOSS Community Day

JAPAN

